

Zaawansowane techniki programowania obiektowego w C++

Zestaw nr 8

1. Zaimplementuj szablon funkcji **integrate**, która jako pierwszy argument przyjmuje dowolny funktor.

Funkcja **integrate** ma dokonywać całkowania w podanym zakresie np.:

```
std::cout<< ::integrate(sina(0),0,3.1415926,0.01)<<std::endl;
```

```
std::cout<< ::integrate(sina(1),0,3.1415926,0.01)<<std::endl;
```

```
std::cout<< ::integrate(sina(2),0,3.1415926,0.01)<<std::endl;
```

2. Stwórz odpowiednie funktory **Variable** oraz **Constant**, które mogą być używane jako argumenty funkcji **integrate**:

```
Variable x;
```

```
integrate(x,0,1,0.001);
```

```
integrate(Constant(1.0),0,1,0.001);
```

3. Funktor **AddExpr** jest używany przez szablon operatora dodawania dwóch funktorów:

```
template<typename LHS,typename RHS >  
AddExpr<LHS,RHS> operator+(const LHS &l,  
const RHS &r)
```

zwracany funktor może być podstawiony do funkcji całkującej. Dokonaj specjalizacji szablonu operatora dodawania dla przypadku, w którym jeden z argumentów jest typu double tak aby możliwe było następujące użycie funkcji **integrate**:

```
integrate(x + 1.,0,1,0.001);
```

4. Zaimplementuj szablony: **operator/(double l, const RHS &r)**, **DivExpr**, **sin** oraz **SinExpr**. Dokonaj całkowania następującej funkcji:

```
integrate(1./sin(x+ 1.,0,1,0.001);
```

5. Operatory arytmetyczne nie będą miały jak automatycznie wydedukować typu T : `template<typename T,typename LHS,typename RHS > Add<T,LHS,RHS> operator+(const LHS &l, const RHS &r)`

ponieważ, typ `T` nie pojawia się w argumentach wywołania. Zaimplementuj dodatkową klasę `Expr` "opakowującą" wyrażenia, która będzie przenosiła informację o typie. Przedstaw przykłady całkowania funkcji z użyciem zmiennej innego typu niż `double`.

6. Zaimplementuj rozszerzenie klasy **Variable** dla przypadku funktorów dwóch zmiennych, tj. klasy **First** oraz **Second**. Podaj przykład całkowania prostej funkcji np. $x + y$.
7. Opisz sposób wykorzystania szablonów wyrażeń wektorowych (na podstawie przygotowanego wcześniej kodu) do efektywnego obliczenia wyrażeń wektorowych np.

```
Vector v1(100,1);
```

```
Vector v2(100,2);
```

```
Vector res(100);
```

```
res = 1.2*v1+v1*v2+v2*0.5;
```

dr Marcin Misiaszek